

Table des matières

1. Introduction	1
1.1. Contexte	1
1.2. Problématique et objectifs	2
1.3. Périmètre du travail et répartition entre étudiants	3
1.4. Démarche suivie	4
2. Analyse et spécification	5
2.1. Fonctionnement général	5
2.2. Acteurs et rôles	7
2.3. Besoins fonctionnels	7
2.4. Besoins non fonctionnels	9
2.5. Règles métier clés	10
2.6. Cas d'utilisation principaux	11
3. Conception de la solution	13
3.1. Architecture générale	13
3.2. Choix technologiques et justification	13
3.3. Modélisation des données	14
3.4. Conception des Training Resources	16
3.5. Authentification et autorisations	17
3.6. Conception de l'API et communication avec le mobile	18
3.7. Fonctionnement hors ligne et synchronisation	19
4. Réalisation de la plateforme web	22
4.1. Mise en place de l'environnement	22
4.2. Implémentation du modèle métier	23
4.3. Implémentation des Training Resources	26
4.4. Implémentation de l'API	27
4.5. Interface utilisateur	29
5. Sécurité et protection des données	31
5.1. Principes de confidentialité et minimisation des données	31
5.2. Sécurité de l'authentification	31
5.3. Isolation des données	31
5.4. Sécurisation de l'API	32
5.5. RGPD et Privacy by Design	32
7.1. Liens avec les cours du bachelier	34

1. Introduction

1.1. Contexte

Ce travail s'inscrit au cœur de l'innovation sportive, plus particulièrement dans le domaine du coaching connecté et des applications mobiles dédiées au sport. Les outils numériques occupent aujourd'hui une place importante dans la pratique sportive, notamment pour accompagner les sportifs dans leur préparation, leur permettre de suivre leurs entraînements et donner aux coaches des moyens d'analyser les performances réalisées.

Dans ce contexte, le projet — nommé NoTrackRun — consiste en une plateforme de coaching sportif destinée à des organisations sportives (clubs, structures sportives) et aux coaches et sportifs qui en dépendent, permettant de faire le lien entre la planification des entraînements, leur réalisation par les sportifs et le suivi de leurs performances. La solution repose sur plusieurs composants complémentaires, notamment une application web destinée à la gestion et au suivi des entraînements ainsi qu’une application mobile utilisée par les sportifs.

Le projet a été développé dans une perspective réelle et commerciale, et non comme une simple démonstration technique réalisée dans le cadre académique. Cette dimension implique de prendre en compte des contraintes propres à un produit destiné à être utilisé par différents utilisateurs, notamment en matière de sécurité, de fiabilité, de confidentialité des données et d’évolution de la solution.

C’est dans ce contexte que Alexandre Henin et moi-même avons commencé à travailler ensemble sur le développement du projet. Nous nous sommes rencontrés au cours de notre bachelier en informatique de gestion et avons décidé de poursuivre cette collaboration autour de cette solution. Le développement avait déjà commencé avant que nous décidions d’en faire le sujet de nos travaux de fin d’études. Le TFE s’inscrit donc dans la continuité d’un projet existant, avec l’objectif d’en approfondir la conception, le développement et les problématiques techniques associées.

L’équipe projet ne se limite toutefois pas aux deux étudiants : la construction du contenu sportif de la plateforme — la structure des plans d’entraînement, la logique d’intensité fondée sur la VMA, la composition des séances — s’appuie sur la collaboration avec un coach expérimenté, qui joue le rôle d’expert métier pour ces aspects. Cette collaboration est développée en 1.4.

Le principe de minimisation des données qui structure l’ensemble de ce travail n’est pas né d’un choix de conception a posteriori : il trouve son origine dans un premier concept d’entraînement “sans traçage” développé initialement par Alexandre Henin dans un contexte militaire, où l’absence de suivi de localisation des personnels répond à un besoin opérationnel concret plutôt qu’à une préférence de confidentialité générale. Ce concept n’ayant pas été retenu dans ce cadre initial, il a ensuite été adapté et étendu au monde civil du coaching sportif — une trajectoire qui n’est pas inhabituelle pour des technologies initialement pensées dans un contexte de défense. Le nom du produit, NoTrackRun, porte directement la trace de cette origine.

Cette situation nous permet également d’aborder le projet avec une approche différente de celle d’un développement réalisé uniquement pour répondre à une demande académique : les choix effectués doivent répondre à des besoins concrets et tenir compte des contraintes d’un produit amené à évoluer.

1.2. Problématique et objectifs

Les applications sportives occupent aujourd’hui une place importante dans le suivi de l’activité physique. Elles permettent notamment d’enregistrer des performances, de suivre la progression d’un sportif et de partager certaines informations avec un coach. Cette utilisation croissante du numérique implique cependant la collecte et le traitement d’un nombre important de données liées aux utilisateurs.

Ces données peuvent avoir un caractère particulièrement sensible. Selon les fonctionnalités proposées, une application sportive peut notamment avoir accès à des informations relatives aux performances, aux habitudes d’entraînement, aux déplacements ou encore à la localisation d’un utilisateur. La multiplication des services sportifs numériques soulève ainsi des questions concernant la manière

dont ces données sont collectées, stockées, utilisées et éventuellement partagées avec des services tiers.

Cette problématique s’inscrit également dans une réflexion plus large autour de la maîtrise des données par les acteurs qui les traitent. Dans un contexte européen marqué par le renforcement des exigences en matière de protection des données, il devient nécessaire de concevoir des solutions permettant de conserver une maîtrise aussi importante que possible des informations traitées — une préoccupation qui se traduit, à l’échelle de ce projet, par des choix concrets de conception plutôt que par une orientation générale.

Le projet cherche donc à se distinguer d’une approche dans laquelle la collecte de données serait systématique simplement parce qu’elle est techniquement possible. L’objectif est au contraire de déterminer quelles informations sont réellement nécessaires au fonctionnement du service, puis de limiter leur collecte et leur accès au strict besoin fonctionnel.

À cela s’ajoute une problématique propre au fonctionnement du produit : le sportif utilise principalement une application mobile pour consulter et réaliser ses entraînements, tandis que le coach utilise la plateforme web pour préparer les séances et suivre les résultats. Les informations doivent donc circuler entre ces deux environnements de manière fiable, y compris lorsque le sportif ne dispose pas d’une connexion réseau au moment où il réalise sa séance.

À partir de ces différents enjeux, la problématique principale de ce travail peut être formulée comme suit :

Comment concevoir et développer une plateforme de coaching sportif permettant de planifier et de suivre les entraînements, tout en assurant une communication fiable avec l’application mobile et une gestion maîtrisée, sécurisée et respectueuse de la vie privée des données des utilisateurs ?

Les principaux objectifs du projet sont dès lors de développer une plateforme permettant de gérer les entraînements et leurs résultats, d’assurer leur communication avec l’application mobile et de fournir aux coachs les outils nécessaires au suivi des performances.

Un objectif important est également de mettre en œuvre une approche de minimisation des données, afin que les informations collectées et accessibles correspondent réellement aux besoins du service. La sécurité, l’isolation des données entre les utilisateurs et les organisations ainsi que la fiabilité de la synchronisation constituent également des objectifs essentiels du développement.

Enfin, le projet doit rester suffisamment évolutif pour pouvoir accompagner les besoins futurs du produit, tout en conservant une architecture permettant de maîtriser les données et les différents flux entre la plateforme et l’application mobile.

1.3. Périmètre du travail et répartition entre étudiants

Le projet étant constitué de plusieurs composants complémentaires, son développement a été réparti entre les deux étudiants afin de permettre une séparation claire des responsabilités tout en conservant une cohérence globale du produit.

Mon travail porte principalement sur la plateforme web et les services backend associés. Cette partie constitue le point central de gestion du système. Elle comprend notamment la gestion des données, la logique métier, la gestion des utilisateurs et des droits d’accès, la gestion des entraînements ainsi que la mise à disposition des fonctionnalités nécessaires à l’application mobile.

Le second volet du projet concerne l'application mobile destinée aux sportifs. Celle-ci permet notamment de récupérer les entraînements, de les consulter et de transmettre les résultats des séances réalisées. Son développement constitue le travail réalisé en parallèle par Alexandre Henin.

Les deux parties ne sont toutefois pas développées indépendamment. Elles doivent communiquer au travers d'une API, qui constitue l'interface entre la plateforme et l'application mobile. La conception de cette communication fait donc partie des éléments communs nécessaires au bon fonctionnement du produit.

Cette répartition permet ainsi de travailler sur deux systèmes distincts tout en conservant un objectif commun : développer une solution de coaching sportif complète. Les choix réalisés d'un côté peuvent avoir des conséquences sur l'autre, ce qui nécessite une coordination régulière, notamment concernant les données échangées, le fonctionnement des entraînements et la transmission des résultats.

1.4. Démarche suivie

Le développement du projet a été réalisé de manière progressive, en partant des besoins identifiés pour construire et faire évoluer la solution. La démarche suivie s'est articulée autour de plusieurs étapes : l'analyse du fonctionnement attendu, la conception de la solution, son développement, puis la validation des fonctionnalités réalisées. Les aspects liés à la sécurité, à la protection des données et à la communication entre la plateforme et l'application mobile ont été pris en compte tout au long du projet.

Cette démarche ne s'est pas construite dans l'isolement des deux étudiants. La structure d'un entraînement, la façon dont un plan s'organise en séances puis en blocs, le rôle central de la VMA dans le calcul des objectifs, ou encore la distinction entre course à pied et renforcement/conditionnement (cf. 2.1, 2.3) relèvent d'un savoir-faire sportif que ni l'un ni l'autre des étudiants ne possédait initialement. Ces aspects ont été définis et validés en collaboration avec un coach expérimenté, qui a agi comme expert métier tout au long du projet : c'est son expérience du terrain qui a guidé la manière dont les Training Resources devaient être structurées pour rester à la fois flexibles et réellement utilisables par d'autres coaches, plutôt qu'une modélisation construite a priori par les développeurs eux-mêmes. Cette collaboration s'est poursuivie au-delà de la phase de conception initiale : le produit étant déjà utilisé en bêta par de vrais clients, certains besoins — comme celui d'un second type d'entraînement orienté conditionnement — ont émergé directement de l'usage réel de la plateforme plutôt que d'une planification initiale exhaustive.

La suite de ce travail reprend cette démarche en présentant successivement le contexte et les besoins du projet, les choix de conception, la réalisation de la plateforme, les aspects liés à la sécurité et à la protection des données, puis les tests et la validation de la solution.

2. Analyse et spécification

Cette partie présente le fonctionnement de NoTrackRun, elle décrit successivement le fonctionnement général du système, les acteurs qui y interviennent et leurs responsabilités respectives, puis formalise ce fonctionnement en besoins fonctionnels et non fonctionnels, en règles métier et en cas d'utilisation. L'objectif est de poser une compréhension commune du problème à résoudre avant d'en présenter la solution technique.

2.1. Fonctionnement général

Pour rappel le projet consiste en une solution de coaching sportif connecté destinée à des organisations sportives, telles que des clubs, et à leurs coachs et runners. Contrairement à une application de suivi sportif destinée directement au grand public, la solution est organisée autour d'une relation entre une organisation, ses coachs et les sportifs qu'ils accompagnent. La plateforme permet ainsi aux coachs de préparer les entraînements, de les attribuer aux runners et de suivre les performances réalisées.

Le fonctionnement général repose sur plusieurs systèmes complémentaires. La plateforme web constitue l'interface principale utilisée par les organisations et les coachs pour gérer les utilisateurs, préparer les entraînements et consulter les résultats. Une API assure la communication avec l'application mobile, utilisée par les runners pour récupérer leurs entraînements et transmettre leurs performances. L'application mobile permet quant à elle au runner de consulter son programme d'entraînement et de réaliser les séances qui lui ont été attribuées.

Gestion des organisations et des accès. Le fonctionnement débute lorsqu'une organisation est intégrée à la plateforme. Lors de cet onboarding, un quota d'accès est défini pour l'organisation, comprenant notamment un nombre d'accès destinés aux coachs et un nombre d'accès destinés aux runners. Les accès sont ensuite créés par l'administrateur de la plateforme conformément à ces quotas.

Une organisation peut ainsi disposer de plusieurs coachs. Chaque coach peut ensuite créer des runners. Un runner est lié à la fois à son organisation et à son coach.

La plateforme prend également en compte différents niveaux de responsabilité. L'administrateur d'une organisation dispose d'un périmètre limité à son club, tandis que certaines fonctionnalités de gestion de ressources relèvent d'un rôle d'administration à l'échelle de la plateforme. Cette distinction permet de séparer la gestion administrative d'une organisation de la gestion globale des ressources utilisées par les coachs.

Création et planification des entraînements. Une fois son accès créé, le coach utilise la plateforme pour préparer les entraînements de ses runners. Il peut créer de nouveaux runners, mais également construire les entraînements à partir des ressources disponibles.

Le système repose sur le concept de Training Resources. Celles-ci permettent de construire progressivement les différents éléments d'un entraînement. Un coach peut créer ses propres ressources et templates, mais également réutiliser des ressources déjà disponibles créées par ses collègues ou par le coach administrateur de la plateforme. Cette possibilité de réutilisation permet d'éviter de reconstruire systématiquement les mêmes éléments et facilite la préparation de programmes d'entraînement cohérents.

Les ressources sont ensuite utilisées pour composer des séances. Plusieurs séances peuvent être regroupées au sein d'un Training Plan. Celui-ci organise les séances selon une structure temporelle

composée de semaines et de jours. Lorsqu'un plan est attribué à un runner, une date de début permet de positionner les séances dans son calendrier d'entraînement.

La VMA (Vitesse Maximale Aérobie) est le KPI central autour duquel s'organise la construction d'un entraînement. Elle est renseignée par le coach au moment où il attribue un plan à un runner, et constitue la donnée de référence à partir de laquelle les objectifs de chaque séance sont déterminés. La grande majorité des blocs qui composent une séance n'expriment pas leur intensité en valeur absolue (une vitesse ou une allure précise), mais en un niveau d'intensité relatif à la VMA du runner. C'est ce couple — intensité planifiée par le coach, VMA propre au runner — qui permet à la plateforme de calculer automatiquement le résultat attendu d'un bloc (l'allure ou l'objectif que le runner doit viser), sans que ni le coach ni le runner n'aient à effectuer ce calcul eux-mêmes. Un même plan, construit une seule fois par le coach, produit ainsi des objectifs concrets différents pour chaque runner auquel il est attribué, proportionnels à leurs capacités respectives.

Cette logique de calcul par la VMA s'applique toutefois à un seul des deux types d'entraînement pris en charge par la plateforme. Le système distingue en effet la course à pied, pour laquelle la VMA sert de référence à l'intensité de la plupart des blocs, et le renforcement / conditionnement physique, orienté vers des exercices dont les objectifs ne s'expriment pas de la même manière (répétitions, charge, exercice précis, plutôt qu'une intensité relative à une vitesse de référence). Ce second type d'entraînement a été effectivement demandé et exploité dès la phase bêta, à la demande d'un client.

Transmission des entraînements au runner. Une fois qu'un plan a été attribué, le runner récupère son entraînement depuis l'application mobile. La communication entre l'application et la plateforme s'effectue au travers de l'API.

Le runner n'est toutefois pas nécessairement connecté en permanence au réseau. L'application est conçue pour pouvoir récupérer les entraînements et permettre leur réalisation dans un contexte où la connexion peut être temporairement indisponible. Cette contrainte est particulièrement importante dans le cadre d'une activité sportive, qui peut être réalisée dans des environnements où la couverture réseau n'est pas garantie.

Après avoir récupéré son programme, le runner peut réaliser une séance même sans connexion. Les résultats sont alors conservés temporairement par l'application et pourront être transmis à la plateforme lorsque la connexion sera à nouveau disponible.

Réalisation et transmission des performances. Après avoir terminé une séance, le runner transmet ses résultats à la plateforme par l'intermédiaire de l'API lorsque cela est possible. Si une connexion réseau est disponible immédiatement, les résultats peuvent être envoyés sans attendre. Dans le cas contraire, l'application attend le rétablissement de la connexion pour effectuer cette transmission.

Les données transmises correspondent aux informations nécessaires au suivi de la séance. Les performances sont notamment associées aux blocs qui composent l'entraînement, ce qui permet de conserver un niveau de détail suffisamment précis pour analyser le déroulement de la séance.

Cette approche permet également de limiter les données transmises à la plateforme. Le suivi des performances ne nécessite notamment pas la transmission systématique des données GPS détaillées du runner. La solution cherche ainsi à fournir les informations nécessaires au coaching tout en limitant la quantité de données personnelles et sportives collectées.

Consultation et analyse des performances. Une fois les résultats reçus par la plateforme, le coach peut consulter les performances de ses runners depuis l'interface web.

Deux niveaux de consultation sont proposés. Le premier correspond à une analyse détaillée de la séance, avec une consultation des résultats bloc par bloc — cette vue permet au coach d'examiner précisément la réalisation de l'entraînement et de comparer les performances obtenues aux éléments prévus dans la séance, la VMA du runner servant de référence commune pour interpréter ces deux valeurs. Le second niveau correspond à une vue plus globale de l'activité : la plateforme propose un dashboard permettant de présenter des statistiques et des graphiques afin de donner au coach une vision synthétique des performances et de l'activité de ses runners.

2.2. Acteurs et rôles

Le système distingue plusieurs acteurs selon leur niveau d'intervention. Cette séparation permet de répartir les responsabilités entre la gestion globale de la plateforme, la gestion d'une organisation, le suivi des runners et l'exécution des entraînements.

Synthèse des responsabilités

Acteur	Périmètre	Responsabilités principales
Administrateur plateforme	Toute la plateforme	Gestion des organisations, quotas et demandes de quotas
Coach administrateur	Toute la plateforme	Création et gestion des Training Resources de base
Administrateur organisation	Une organisation	Gestion des coaches, utilisateurs et besoins de l'organisation
Coach	Son organisation / ses runners	Création des runners, entraînements, plans et suivi des résultats
Runner	Ses propres entraînements	Réception des plans, exécution des séances et envoi des résultats

2.3. Besoins fonctionnels

Les besoins fonctionnels décrivent les principales capacités que le système doit fournir aux différents acteurs. Ils permettent de définir ce que chaque utilisateur doit pouvoir réaliser au sein de la plateforme, sans encore préciser les conditions ou contraintes qui encadrent ces actions.

Acteur	Besoin fonctionnel	Description
Administrateur de la plateforme	Gérer les organisations	Créer et gérer les organisations présentes sur la plateforme.
Administrateur de la plateforme	Gérer les quotas	Définir et modifier les quotas d'accès aux coaches et aux runners des organisations.

Acteur	Besoin fonctionnel	Description
Coach administrateur	Gérer les ressources de base	Créer et maintenir les ressources d'entraînement mises à disposition des coaches.
Administrateur d'organisation	Gérer les coaches	Créer, supprimer et gérer les coaches de son organisation.
Administrateur d'organisation	Gérer les utilisateurs	Gérer les utilisateurs de son organisation et leurs affectations.
Administrateur d'organisation	Gérer les besoins en accès	Demander une modification du quota de son organisation.
Coach	Gérer les runners	Créer et gérer les runners dont il assure le suivi.
Coach	Gérer les Training Resources	Créer et utiliser des ressources d'entraînement.
Coach	Créer des séances	Composer les séances d'entraînement à partir des ressources disponibles, pour un ou plusieurs types d'entraînement (course à pied, renforcement/conditionnement).
Coach	Gérer les Training Plans	Créer et organiser les plans d'entraînement.
Coach	Attribuer des plans	Attribuer un Training Plan à un runner, en renseignant sa VMA de référence.
Coach	Consulter les résultats	Consulter les performances réalisées par ses runners, au regard des objectifs calculés à partir de leur VMA.
Coach	Consulter les statistiques	Consulter une vue globale des performances au moyen de statistiques et de graphiques.
Runner	Récupérer ses entraînements	Récupérer les plans et séances qui lui sont attribués.
Runner	Réaliser ses séances	Consulter et réaliser les séances depuis l'application mobile.
Runner	Transmettre ses résultats	Envoyer les résultats des séances réalisées vers la plateforme.

Ces besoins représentent les principales capacités attendues du produit. Ils permettent de couvrir l'ensemble du cycle fonctionnel, depuis la gestion des organisations jusqu'au suivi des performances des runners.

2.4. Besoins non fonctionnels

En complément des fonctionnalités précédemment identifiées, la plateforme doit respecter plusieurs exigences non fonctionnelles. Celles-ci concernent principalement la sécurité, la confidentialité des données, la disponibilité du service, la gestion des situations hors ligne ainsi que la maintenabilité et l'évolutivité du système.

Catégorie	Besoin non fonctionnel	Description
Sécurité	Contrôle des accès	L'accès aux fonctionnalités et aux données doit être limité en fonction du rôle de l'utilisateur et de son organisation.
Sécurité	Authentification sécurisée	Les utilisateurs et les échanges avec l'API doivent être authentifiés de manière sécurisée.
Sécurité	Isolation des organisations	Les données d'une organisation ne doivent pas être accessibles aux utilisateurs d'une autre organisation.
Confidentialité	Minimisation des données	La plateforme doit limiter la collecte et la transmission des données personnelles aux informations nécessaires au fonctionnement du service.
Confidentialité	Protection de l'identité du runner	Les informations personnelles du runner doivent être limitées. Le coach utilise notamment un pseudo plutôt qu'une identité personnelle complète.
Disponibilité	Accessibilité du service	La plateforme et son API doivent rester disponibles afin de permettre aux applications mobiles de récupérer les entraînements et de transmettre les résultats.
Hors ligne	Tolérance à l'absence de réseau	Le fonctionnement du runner ne doit pas dépendre d'une connexion réseau permanente.

Catégorie	Besoin non fonctionnel	Description
Synchronisation	Transmission différée	Les résultats produits hors connexion doivent pouvoir être transmis ultérieurement lorsque le réseau est disponible.
Fiabilité	Intégrité des résultats	Les résultats reçus après une période hors ligne doivent pouvoir être correctement associés aux éléments de l'entraînement auxquels ils correspondent.
Maintenabilité	Évolution du système	L'architecture doit permettre de faire évoluer les fonctionnalités sans remettre en cause l'ensemble de la plateforme.
Évolutivité	Gestion de plusieurs organisations	Le système doit pouvoir accueillir plusieurs organisations avec leurs propres utilisateurs, ressources et données sans mélanger leurs périmètres.
Flexibilité	Construction des entraînements	Le système doit permettre aux coaches de créer et de réutiliser différentes Training Resources afin de construire des entraînements adaptés à leurs besoins.

Ces exigences sont particulièrement importantes dans le contexte du projet, car la plateforme doit assurer la communication entre plusieurs types d'utilisateurs et une application mobile pouvant fonctionner temporairement sans connexion — les contraintes de fonctionnement hors ligne et de minimisation des données, déjà présentées en 2.1, se retrouvent ici formalisées en exigences non fonctionnelles à part entière.

2.5. Règles métier clés

Les règles métier définissent les contraintes qui encadrent l'utilisation des fonctionnalités du système. Elles précisent notamment les relations entre les utilisateurs, les conditions d'accès aux ressources et le comportement attendu lors de situations particulières.

Organisation / quotas

- Une organisation possède un quota d'accès coaches et un quota d'accès runners.
- L'administrateur de la plateforme crée l'organisation et gère ses quotas.
- L'administrateur d'organisation peut demander une augmentation ou une modification du quota.
- Un coach ne peut être rattaché qu'à une seule organisation.
- Une organisation peut avoir plusieurs coaches.

Utilisateurs

- Un runner appartient à une organisation.
- Un runner est également rattaché à un coach.
- Si un coach est supprimé, le runner reste dans l'organisation et peut être réaffecté à un autre coach.
- L'administrateur d'organisation peut créer, supprimer et réaffecter les coaches et les utilisateurs de son organisation.

Training Resources

- Il existe des Training Resources de base, créées par le coach administrateur et disponibles pour les coaches de toutes les organisations.
- Un coach peut utiliser ces ressources de base.
- Un coach peut également créer ses propres Training Resources.
- Les ressources créées par un coach sont automatiquement partagées avec les autres coaches de la même organisation.
- Une Training Resource existante ne peut pas être modifiée directement par un coach s'il n'en est pas le créateur. Pour l'adapter, celui-ci doit en créer une copie, puis modifier cette copie. Cette règle s'applique également aux ressources de base.

Plans / entraînements

- Un coach crée des séances à partir des Training Resources.
- Il peut regrouper ces séances dans un Training Plan.
- Il peut attribuer un Training Plan à un runner, en renseignant sa VMA.
- Une fois attribué, le runner peut récupérer le plan via l'application mobile.

Résultats / hors ligne

- Le runner peut exécuter un plan sans connexion après l'avoir récupéré.
- Les résultats sont envoyés dès que la connexion est disponible.
- Les résultats sont associés aux blocs de la séance.
- Un bloc peut apparaître plusieurs fois dans les résultats, notamment lorsqu'il comporte des répétitions.
- Si le coach modifie ou supprime un bloc ou une séance pendant que le runner est hors ligne, les résultats du runner doivent quand même pouvoir être associés correctement à ce qu'il avait exécuté.

Permissions / isolation

- Une organisation ne peut accéder qu'à ses propres données.
- Un coach ne voit que les runners relevant de son périmètre.
- L'administrateur d'organisation peut avoir une vue globale sur l'activité de son organisation.
- Le coach administrateur peut créer des ressources globales utilisables par les différentes organisations.

2.6. Cas d'utilisation principaux

Les cas d'utilisation permettent de représenter les principales interactions entre les acteurs et la plateforme. Ils complètent les besoins fonctionnels en mettant en évidence les actions réalisées par chaque type d'utilisateur au sein du système.

DIAGRAMME CAS D'UTILISATION

3. Conception de la solution

Cette partie présente les choix de conception effectués pour répondre aux besoins et contraintes formalisés en partie 2. Elle part de l'architecture générale du système pour descendre progressivement vers les décisions les plus spécifiques : la modélisation des données, l'abstraction des Training Resources, l'authentification, la communication avec l'application mobile, et enfin la gestion du fonctionnement hors ligne.

3.1. Architecture générale

La plateforme repose sur une architecture monolithique : un serveur unique Laravel héberge à la fois l'application web (destinée aux organisations et aux coachs) et l'API consommée par l'application mobile (destinée aux runners). Il n'y a pas de séparation physique entre ces deux façades, mais une séparation logique au niveau du routage et de l'authentification : un ensemble de routes sert l'interface web, un autre expose une API JSON dédiée au mobile, avec des mécanismes d'authentification distincts pour chacune (session côté web, token côté mobile — détaillé en 3.5).

Ce choix se justifie par le contexte du projet : une équipe restreinte (un développeur par partie), un produit en phase bêta commerciale, et un besoin de limiter la complexité opérationnelle. Une architecture distribuée (API séparée, services indépendants) aurait introduit une charge d'infrastructure et de coordination disproportionnée par rapport au stade actuel du produit. Le compromis assumé est un couplage plus fort entre web et mobile au sein d'une même base de code, en échange d'une simplicité de déploiement et de maintenance.

Les deux façades partagent la même couche métier (modèles, règles de permission, logique applicative), ce qui évite la duplication de logique entre elles — bénéfice concret du choix monolithique dans ce contexte.

DIAGRAMME ARCHI

3.2. Choix technologiques et justification

Laravel 12 / PHP 8 et MariaDB n'ont pas fait l'objet d'une étude comparative formelle. Ils restent défendables au regard du besoin : le modèle de données du projet est fortement relationnel, sans exigence particulière qui aurait nécessité un autre SGBD ou un autre framework backend.

Inertia.js + Vue.js, en revanche, se justifie plus substantiellement à l'usage. Sans Inertia, il aurait fallu construire une véritable SPA consommant une API REST dédiée au web — donc maintenir deux API distinctes (web et mobile), avec deux logiques de sérialisation et deux couches d'autorisation à faire évoluer en parallèle. Avec Inertia, le serveur reste seul maître de l'état : chaque navigation renvoie directement un composant Vue avec ses données, sans exposer de JSON public réutilisable côté web. Seule l'API mobile reste alors une véritable API à concevoir et sécuriser comme telle (cf. 3.6).

C'est un compromis assumé : Inertia couple le frontend web aux routes Laravel, ce qui est acceptable puisque cette interface n'a pas vocation à être consommée autrement que par le navigateur — contrairement au mobile, qui nécessite une véritable API découplée.

L'authentification s'appuie sur Laravel Sanctum, complété par un système de tokens propre au projet ; ce choix est développé en 3.5, où il est directement lié aux contraintes du fonctionnement hors ligne du mobile.

3.3. Modélisation des données

Hiérarchie des acteurs. Le modèle relationnel confirme et précise la hiérarchie organisation → coach → runner décrite en 2.5, chaque niveau étant modélisé comme une entité distincte plutôt que comme une variante d'un modèle utilisateur unique.

L'organisation dispose de ses propres identifiants de connexion, distincts de ceux de ses coaches — l'acteur « organisation » n'est donc pas un coach disposant d'un privilège supplémentaire, mais le compte du club lui-même. Elle porte également les quotas contractuels convenus à l'onboarding.

NoTrackRun elle-même est représentée comme une organisation, distinguée des organisations clientes par un indicateur dédié — on la désigne ci-après comme l'organisation admin. C'est en son sein que sont rattachés les comptes des coaches admin : des coaches classiques dans leur structure, mais dont le rattachement à l'organisation admin plutôt qu'à une organisation cliente leur confère le rôle plateforme décrit en 2.5. Le rôle plateforme est ainsi traité comme un cas particulier du modèle organisation/coach existant, plutôt que comme une exception à celui-ci.

Le runner, enfin, est le seul acteur sans identifiants de connexion propres à la plateforme web : il s'authentifie uniquement depuis l'application mobile (cf. 3.5), et ne porte aucune donnée d'identité au-delà d'un pseudonyme — matérialisation concrète du principe de minimisation énoncé en 5.1. Sa double relation à son coach et à son organisation permet — conformément à la règle métier de 2.5 — de le conserver au sein de l'organisation si son coach est supprimé, en vue d'une réaffectation ultérieure sans perte de son historique.

DIAGRAMME USERS

Modélisation d'un plan d'entraînement : training plan vs user training plan. Le plan d'entraînement est modélisé en deux entités distinctes, qui répondent chacune à un besoin différent. Cette distinction est le choix de conception le plus structurant de tout le module de planification, et mérite d'être développée.

Le training plan est une ressource de coaching réutilisable, au même titre qu'un type de bloc ou un template de séance : il appartient à un coach, et ne fait référence à aucun runner en particulier. Ses séances ne sont pas positionnées à des dates fixes, mais à des coordonnées relatives — un numéro de semaine et un numéro de jour, formant un calendrier relatif qui peut être réutilisé pour n'importe quel runner, à n'importe quel moment. C'est cette structure relative qui permet à un même plan-modèle d'être assigné à plusieurs runners différents, à des dates de début différentes, sans avoir à en dupliquer la définition.

Le user training plan est créé au moment précis où un coach attribue un plan-modèle à un runner, avec une date de début choisie. À cet instant :

- le plan-modèle et l'ensemble de ses séances sont copiés dans une structure propre à ce runner, plutôt que simplement référencés ;
- la coordonnée relative (semaine/jour) de chaque séance copiée est conservée telle quelle, permettant de la replacer dans le calendrier réel du runner à partir de sa date de début propre.

Cette duplication, plutôt qu'une simple relation par référence, répond à trois besoins distincts :

1. **Indépendance vis-à-vis des modifications ultérieures du plan-modèle.** Si le coach fait évoluer le plan-modèle après l'avoir déjà assigné à un ou plusieurs runners, ces modifications ne doivent pas se répercuter rétroactivement sur ce que les runners ont déjà reçu — chacun garde la version qu'il avait au moment de l'assignation.

2. **Divergence indépendante par runner.** Deux runners assignés au même plan-modèle peuvent ensuite diverger l'un de l'autre : feedback, commentaires, suppressions ponctuelles d'une séance pour l'un sans affecter l'autre, alors qu'ils partagent la même origine.
3. **Cohérence avec la contrainte de fonctionnement hors ligne.** Le plan assigné constitue la référence stable côté serveur de ce que le runner a reçu, indépendamment des évolutions ultérieures du plan-modèle — un point directement réutilisé dans la réflexion sur la synchronisation développée en 3.7.

Composition d'un plan d'entraînement. Qu'il s'agisse d'un plan-modèle ou d'un plan assigné, la structure interne d'un plan suit la même hiérarchie d'entités :

DIAGRAMME ENTITES PLAN

Entité	Rôle dans le plan
Plan	Regroupe l'ensemble des séances d'un programme d'entraînement, positionnées dans un calendrier relatif (semaine/jour).
Session	Une séance d'entraînement à réaliser un jour donné du plan ; regroupe les blocs qui la composent.
SessionType	Catégorise la séance (ex. Endurance, VMA, Récupération), indépendamment de son contenu détaillé.
Block	Unité de travail au sein d'une séance (ex. échauffement, intervalle, récupération) ; peut en contenir d'autres (besoin métier spécifique).
BlockType	Définit la nature d'un bloc (ex. Interval, Récupération) et détermine quels champs lui sont associés.
BlockField	Valeur planifiée par le coach pour un champ donné d'un bloc (ex. distance : 1000).
Unit	Unité de mesure dans laquelle une valeur de champ est exprimée (ex. mètres, minutes).
BlockResult	Résultat réellement exécuté par le runner pour un bloc (distance parcourue, durée, réussite), distinct de la valeur planifiée.

Une séance est composée d'un ou plusieurs blocs, ordonnés. Chaque bloc correspond à une occurrence concrète d'un type de bloc, c'est ce type qui détermine quels champs sont pertinents pour ce bloc : un bloc de type « Interval » attend par exemple une distance, une répétition et une intensité, alors qu'un bloc de type « Récupération » n'attend qu'une durée et une intensité. Chaque bloc porte ainsi les valeurs concrètes de ces champs pour cette occurrence précise : c'est la donnée planifiée par le coach, ce que le runner est censé réaliser.

Les blocs peuvent en outre être imbriqués : un bloc peut en contenir d'autres, ce qui répond à un

besoin métier précis — permettre au coach d’insérer des accélérations ponctuelles à l’intérieur d’un bloc (par exemple une accélération de quelques secondes au sein d’un bloc d’endurance), sans avoir à modéliser cela comme une séquence de blocs indépendants au même niveau que le bloc qui les contient. Dans ce cas, c’est le bloc parent qui porte ses propres champs planifiés (par exemple la distance et l’intensité globales), tandis que ses sous-blocs représentent ces éléments ponctuels de façon indicative, sans être aujourd’hui individuellement suivis en résultat — seul le bloc parent fait l’objet d’un enregistrement de résultat. Cet usage de l’imbrication reste circonscrit à ce besoin pour l’instant, mais la structure sous-jacente ne l’y restreint pas : elle pourra être exploitée pour d’autres formes de composition à mesure que le besoin métier évolue.

DIAGRAMME SESSION

3.4. Conception des Training Resources

Le problème : des règles transversales à plusieurs types de ressources. Un `BlockType`, un `SessionType`, une séance-type ou un plan sont des objets métier différents, mais qui posent tous exactement les mêmes questions : qui en est propriétaire, qui peut le consulter, le modifier, le supprimer, le copier, l’utiliser dans une séance ? Sans mutualisation, chacune de ces règles devrait être réécrite pour chaque type de ressource — avec le risque, à chaque évolution de la règle, de devoir la répercuter dans plusieurs classes en parallèle, au risque d’incohérences.

La plateforme répond à ce problème par une abstraction commune, `TrainingResource`, dont héritent l’ensemble des ressources de coaching réutilisables. Chaque type concret (`SessionType`, `BlockType`, séance-type, plan...) n’implémente que ce qui lui est propre ; la logique de propriété et de permission, elle, est écrite une seule fois.

Une abstraction organisée par comportement, pas par type. Plutôt qu’une seule interface générique, `TrainingResource` implémente plusieurs petites interfaces, chacune correspondant à un comportement précis : être visualisable, modifiable, supprimable, copiable, utilisable, et avoir un propriétaire. Ce découpage a un intérêt concret : du code générique (par exemple, la sérialisation des droits envoyée au frontend) peut raisonner sur « est-ce que cet objet est modifiable ? » sans jamais avoir besoin de savoir s’il s’agit d’un `BlockType` ou d’un plan. La méthode `permissions()`, définie une seule fois dans la classe de base, calcule ainsi le même ensemble de droits (voir/éditer/supprimer/copier/utiliser) pour n’importe quelle ressource, et cette structure est directement exploitée côté interface pour déterminer ce que peut faire l’utilisateur connecté.

Propriété et permissions calculées, non stockées. Aucune règle de permission n’est stockée comme attribut figé d’une ressource : tout est déduit au moment de la requête, à partir de deux éléments — le propriétaire de la ressource, et l’identité de l’utilisateur qui la consulte.

La propriété elle-même se décline à deux niveaux : une ressource appartient soit à un coach précis, soit directement à l’organisation (lorsqu’aucun coach n’y est rattaché). Un coach ne peut modifier ou supprimer qu’une ressource dont il est lui-même le créateur ; une organisation peut seulement réattribuer une ressource sans coach. La lecture, en revanche, est ouverte à l’ensemble des coaches d’une même organisation par défaut — ce qui confirme et rend concret le fonctionnement décrit en 2.5 : le partage au sein d’une organisation n’est pas une option à activer, c’est le comportement de base du système.

Concilier protection et flexibilité : la copie. La protection en modification pose un problème pratique : que faire si un coach souhaite partir d’une ressource protégée pour construire sa propre variante ? C’est le rôle du comportement « copiable » : il permet à un coach de dupliquer n’importe

quelle ressource qu’il peut consulter, la copie lui appartenant alors en propre et devenant pleinement modifiable. La protection ne verrouille donc jamais l’usage d’une ressource, seulement sa modification directe — un coach reste toujours libre d’en repartir pour construire la sienne.

Un second modèle de permission pour les ressources assignées. L’abstraction ne s’arrête pas aux ressources réutilisables du coach. Les entités assignées à un runner (plan assigné, séance assignée — cf. 3.3) héritent également de `TrainingResource`, via une seconde classe intermédiaire qui redéfinit entièrement la logique de permission : l’accès n’y dépend plus de la propriété d’une ressource par un coach ou une organisation, mais de la relation d’encadrement entre le coach consultant et le runner concerné.

Ce choix illustre l’intérêt de l’abstraction par capacités plutôt que par héritage rigide : deux modèles de permission entièrement différents — l’un fondé sur la propriété et le partage, l’autre sur la relation coach/runner — cohabitent sous un même contrat, sans dupliquer la structure générale (calcul des droits, sérialisation vers le frontend) qui, elle, reste commune aux deux.

Une ressource protégée au service d’une logique algorithmique. Le statut de ressource protégée ne sert pas qu’à restreindre l’édition : il garantit aussi qu’une ressource exploitée par une logique algorithmique de la plateforme ne peut être altérée par un coach. C’est le cas de la séance-type utilisée pour le test de VMA, dont les résultats alimentent un calcul automatique — sa non-modifiabilité est une nécessité pour la fiabilité de ce calcul. Cette séance fait partie d’un socle de ressources de base fournies avec la plateforme dès l’onboarding d’une organisation, aux côtés d’autres ressources standards.

3.5. Authentification et autorisations

Cette section présente le fonctionnement du système d’authentification et la raison d’être de chacun de ses éléments dans le parcours utilisateur. Les aspects proprement sécuritaires sont approfondis en 5.2, pour ne pas dupliquer le propos.

Un flow d’authentification différent selon l’acteur. Organisation et coach s’authentifient tous deux par un couple identifiant/mot de passe classique, le fonctionnement ne diffère pas d’une application web standard. Le runner, en revanche, n’a ni identifiant ni mot de passe (cf. 3.3, minimisation des données) : son parcours d’authentification est entièrement différent, et repose sur trois éléments distincts plutôt qu’un seul mécanisme.

Le parcours du runner : de la création à l’usage courant. L’onboarding suit la hiérarchie déjà posée en 3.3 : l’organisation crée ses coaches, puis un coach crée un runner en lui attribuant simplement un pseudonyme — aucune information de contact ni mot de passe n’est demandée à ce stade.

1. **Activation.** À la création du runner, le système génère un token d’activation, à usage unique, que le coach transmet au runner. Celui-ci l’utilise une seule fois, au premier lancement de l’application mobile, pour rattacher l’appareil à son compte runner — c’est ce token qui remplace l’étape classique d’inscription par identifiant/mot de passe.
2. **Token d’accès.** L’activation réussie, l’application reçoit un token d’accès (durée de vie de 14 jours), conservé sur l’appareil. C’est ce token qui permet au runner de rester authentifié sur une longue période sans avoir à ressaisir quoi que ce soit — une nécessité directe du fonctionnement hors ligne prolongé décrit en 3.7.
3. **Token de session.** Pour chaque action effective sur la plateforme — récupérer un plan, envoyer des résultats — l’application présente son token d’accès afin d’obtenir un token de

session Sanctum, de durée de vie beaucoup plus courte (15 minutes) ; c'est ce dernier qui authentifie réellement les appels à l'API. Le token d'accès est lui-même à usage unique : chaque présentation déclenche, en plus du nouveau token de session, la génération d'un nouveau token d'accès qui remplace le précédent — une rotation glissante plutôt qu'un token fixe consommé pendant toute sa durée de vie. La fenêtre de 14 jours se prolonge ainsi à chaque usage effectif de l'application, plutôt que de courir immuablement depuis l'activation initiale.

Que se passe-t-il à l'expiration du token d'accès ? Passé les 14 jours, le token d'accès expire sans renouvellement automatique : le runner ne peut plus obtenir de nouveau token de session, et donc plus interagir avec la plateforme. Il doit alors s'adresser à son coach pour qu'un nouveau token d'activation lui soit généré, reproduisant la première étape du parcours. Côté web, le coach est alerté lorsqu'un token de l'un de ses runners arrive à expiration.

3.6. Conception de l'API et communication avec le mobile

Cette section porte sur les principes de conception de l'API mobile — la nature des échanges et la forme des données — et non sur son implémentation (routage, middlewares...), traitée en 4.4.

Endpoint	Rôle
POST /users/activate	Active l'application avec le token d'activation (cf. 3.5)
GET /user/training-plans	Récupère l'ensemble des plans assignés au runner
POST /user/session-result/{session}	Envoie les résultats d'une séance exécutée
POST /user/session-review/{session}	Envoie le feedback qualitatif du runner sur une séance
POST /user/bug-report	Permet de signaler un problème directement depuis l'application

Le dernier endpoint mérite une remarque : il illustre concrètement la nature de produit commercial en production du projet, plutôt qu'un exercice académique — un canal de retour existe pour que les runners signalent un dysfonctionnement directement depuis l'application.

Un seul appel, un payload consolidé. GET /user/training-plans ne renvoie pas un plan à la fois, mais l'ensemble des plans actuellement assignés au runner, chacun avec l'intégralité de sa hiérarchie déjà résolue — séances, blocs, champs planifiés et résultats déjà enregistrés. Le mobile n'a donc aucune reconstruction à faire à partir de fragments : un seul appel suffit à obtenir tout ce dont l'application a besoin pour fonctionner hors ligne pendant une période prolongée (cf. 3.7).

Un détail de conception important : les séances sont transmises avec une date absolue déjà calculée (`"date": "2026-03-10"`), et non le couple semaine/jour relatif utilisé en base (cf. 3.3). C'est le serveur qui résout ce calcul à partir de la date de début du plan assigné, avant l'envoi — le mobile n'a ainsi aucune logique de calendrier à reproduire, ce qui réduit d'autant la surface de désynchronisation possible entre les deux implémentations.

Payload réel complet en annexe 10.6.

Transmission des résultats. À l'inverse, l'envoi des résultats se fait séance par séance plutôt que plan entier, avec un payload volontairement minimal — un point déjà présenté en amont du

document (index, distance, durée, réussite, identifiant du bloc concerné), qui correspond directement à la structure `BlockResult`. Cette asymétrie entre un payload de lecture riche et complet et un payload d'écriture minimal reflète la contrainte de minimisation des données.

Format de réponse et validation. Toutes les réponses de l'API suivent une enveloppe commune (success, error, data, status), qu'il s'agisse d'un succès ou d'une erreur métier — un principe de conception simple qui uniformise le traitement des réponses côté mobile, indépendamment de l'endpoint appelé. Les règles de validation des données entrantes (formats attendus, contraintes métier) sont, elles, définies au niveau de la conception mais mises en œuvre techniquement en 4.4 ; leur dimension sécuritaire est développée en 5.4.

3.7. Fonctionnement hors ligne et synchronisation

Le problème. Le runner récupère (pull) son plan puis peut rester hors ligne pendant une période prolongée — plusieurs semaines dans certains cas — avant de renvoyer (push) ses résultats (cf. 3.6). Pendant cette période, rien n'empêche le coach de continuer à faire évoluer ce même plan côté serveur : modifier, voire supprimer, une séance ou un bloc que le runner a déjà en local et est en train — ou a déjà fini — d'exécuter.

Le système doit donc composer avec une divergence possible entre l'état du plan tel que le serveur le connaît au moment du push, et l'état tel que le runner l'a réellement exécuté, sans pour autant perdre les résultats transmis ni compromettre l'intégrité de la base.

DIAGRAMME FLOW SYNC

La solution actuelle : suppression logique et restauration. Pour garantir que les résultats du runner puissent toujours être associés à un bloc et à une séance valides en base, le système repose sur une suppression logique (soft delete) plutôt que physique : une séance ou un bloc « supprimé » par le coach n'est pas réellement effacé, et se trouve restauré automatiquement dès que des résultats le concernant arrivent — l'intégrité référentielle du côté du push reste ainsi toujours garantie, quel que soit l'état dans lequel le coach a laissé le plan entre-temps.

Ce mécanisme présente deux propriétés notables :

- **Il opère en cascade.** Restaurer un bloc restaure également les entités qui lui sont directement liées (champs planifiés, et selon le type de bloc, ses autres composants) — la restauration ne se limite pas à la ligne du bloc lui-même, elle rétablit tout ce qui est nécessaire pour que le résultat transmis puisse s'y rattacher de façon cohérente.
- **Il s'accompagne d'un garde-fou contre les doubles envois.** Un résultat n'est enregistré pour un bloc que si celui-ci n'en possède pas déjà en base au moment du traitement de la requête — une protection utile dans un contexte de connectivité mobile dégradée, où l'application peut être amenée à retenter un push resté sans confirmation, sans risquer de dupliquer les résultats déjà bien reçus.

Ce mécanisme de restauration est aujourd'hui déclenché explicitement dans le contrôleur qui traite la réception des résultats, plutôt que par un mécanisme centralisé et générique. C'est un choix qui fonctionne, mais qui n'est pas neutre — on y revient plus loin, car c'est précisément le type de risque qui a pesé dans l'abandon d'une première tentative de résoudre un problème voisin.

Limite actuelle : une traçabilité insuffisante. Le plan assigné porte deux horodatages — dernier pull, dernier push (cf. 3.3) — qui indiquent quand le runner a synchronisé, mais pas quelle version exacte du plan il avait alors en main. Si le coach modifie un plan après qu'un runner l'a récupéré,

rien aujourd’hui ne permet de reconstituer précisément ce que ce dernier a effectivement reçu et exécuté, au-delà de ce qu’il a bien voulu renvoyer comme résultats.

Comblé cette limite suppose de conserver, d’une manière ou d’une autre, l’état du plan tel qu’il était à un instant donné — une préoccupation différente de la suppression logique déjà en place (qui ne couvre que les suppressions, pas les modifications de valeur). C’est ce besoin qui a motivé une première tentative, testée puis abandonnée, avant qu’une piste alternative ne soit retenue.

Tentative abandonnée : un event sourcing hybride. Une première approche par event sourcing a été explorée : des événements (`UpdateUserSessionBlockEvent`, etc.) étaient déclenchés manuellement dans les contrôleurs à chaque modification d’un bloc. Chaque événement portait un payload contenant l’état complet et dénormalisé du bloc concerné à cet instant — libellé, couleur, champs, unités, icônes — associé à un utilisateur, une entité et un type d’événement.

Ce n’était pas un event sourcing « pur » : le payload de chaque événement contenait déjà un état complet d’une seule entité, et non un delta compact. C’était un hybride, empruntant la granularité fine d’un log d’événements sans le bénéfice structurel d’un vrai event sourcing (deltas compacts, replay incrémental).

Deux problèmes concrets ont motivé l’abandon de cette piste :

1. **Un décalage avec le format attendu côté mobile.** Le mobile attend un payload consolidé — le plan entier, en un seul lot (cf. 3.6) — pas une série de micro-événements à rejouer. Reconstruire « ce que le runner a reçu » aurait nécessité de retrouver tous les événements pertinents antérieurs à la date de pull, de ne garder que le dernier par bloc, puis de réassembler manuellement toute la hiérarchie plan → séance → bloc. Un travail de reconstruction en lecture plus lourd que ce que le pattern est censé apporter, sans bénéficier de son intérêt réel.
2. **Un déclenchement dispersé dans les contrôleurs.** Les événements étaient créés manuellement à chaque point d’entrée touchant un bloc — ce qui suppose de penser à les déclencher partout où c’est pertinent (mise à jour directe, suppression en cascade, actions admin...), avec un risque de capture incomplète si un point d’entrée est oublié. C’est exactement le même risque que celui identifié plus haut pour la restauration actuelle, aujourd’hui déclenchée dans un unique contrôleur : la fiabilité d’un mécanisme de traçabilité dépend d’une capture garantie et centralisée, pas dispersée au gré des contrôleurs.

L’event sourcing, même rendu plus « pur », aurait aggravé le problème plutôt que de le résoudre : le besoin côté mobile est grossier (un état consolidé par pull), alors que le pattern vise une granularité fine adaptée à l’audit détaillé, pas à la reconstruction rapide d’un gros payload cohérent.

Piste retenue : reconstruction par date (SCD Type 2). Deux options restent ouvertes pour combler la limite de traçabilité, sans reproduire l’erreur de la tentative précédente.

Le snapshot matérialisé consisterait à dupliquer, à chaque pull, le payload consolidé exactement tel qu’envoyé au mobile — pas de reconstruction ni de replay, juste une copie figée. C’est l’option la plus simple et la plus sûre à mettre en œuvre.

La reconstruction par date, retenue comme piste de conception à approfondir, consiste à ne rien dupliquer et à reconstituer l’état du plan à la demande, en filtrant les données actuelles à partir de la dernière date de pull du runner. Il s’agit d’un principe de SCD (Slowly Changing Dimension) Type 2 — à distinguer explicitement de l’event sourcing testé plus haut : on y stocke des versions d’état bornées dans le temps (`valid_from` / `valid_to`), et connaître l’état à une date T se fait par une simple requête filtrée, sans calcul ni jeu. C’est précisément ce que la tentative event sourcing n’était

pas, malgré une structure de stockage en apparence proche (un log horodaté) : chaque événement y portait déjà un état complet plutôt qu'un delta, ce qui imposait quand même un filtrage puis une déduplication en lecture — une reconstruction plus lourde qu'un simple filtre SCD, sans les bénéfices structurels de l'un ou l'autre pattern.

Pour les suppressions, le soft delete déjà en place (décrit ci-dessus) constitue une brique directement réutilisable pour ce filtre par date. Pour les modifications, en revanche, rien n'existe encore : une mise à jour classique écrase la valeur précédente, qui devient irrécupérable pour une reconstruction a posteriori — il faudrait faire évoluer les écritures vers un versionnement, au minimum sur les entités concernées par la reconstruction.

Point de vigilance. Le risque de capture dispersée déjà identifié pour la restauration actuelle et pour la tentative event sourcing s'appliquerait de la même façon à un versionnement SCD des modifications s'il était, lui aussi, déclenché au coup par coup dans chaque contrôleur. Centraliser la logique d'écriture (service dédié, ou observer Eloquent sur le modèle) plutôt que de la disperser est une condition de fiabilité du mécanisme, pas un détail secondaire — un point à traiter dès la conception plutôt qu'à corriger après coup.

Décision retenue à court terme : snapshot borné, avec purge des périodes inactives. La décision effectivement retenue à court terme combine les deux options plutôt que de trancher entre elles. Le principe reste celui du snapshot matérialisé — le payload consolidé est dupliqué tel quel à chaque pull, sans reconstruction ni rejeu — mais chaque snapshot est désormais borné dans le temps par une période de validité (`valid_from` / `valid_to`), à la manière d'un SCD Type 2 : au moment où un nouveau pull donne lieu à un nouveau snapshot (si le plan a changé depuis le précédent), la période de validité du snapshot précédent est close à cette date, et celle du nouveau démarre.

Une règle de purge vient s'ajouter à ce principe : lorsqu'un nouveau snapshot est créé, si aucun résultat n'a été transmis par le runner pendant toute la période de validité du snapshot précédent, ce dernier est supprimé plutôt que conservé. L'idée est simple : un snapshot qui n'a jamais servi de référence pour un résultat réellement exécuté n'apporte aucune valeur d'audit — il ne fait que représenter une période où le runner a peut-être ouvert l'application sans rien exécuter (un pull répété sans usage réel, par exemple). Ne conserver que les snapshots qui ont effectivement « couvert » une activité du runner permet de garder une trace utile pour le coach, sans laisser le stockage croître au rythme de chaque pull, y compris ceux qui ne correspondent à aucun usage concret.

4. Réalisation de la plateforme web

Cette partie descend au niveau de l'implémentation les choix de conception présentés en partie 3 : environnement technique, modèle métier, Training Resources, API et interface utilisateur. Là où la partie 3 restait volontairement au niveau des principes et des décisions structurantes, il s'agit ici de montrer comment ces décisions se traduisent concrètement en code — migrations, modèles Eloquent, Politiques, middlewares, contrôleurs et composants Vue — en réutilisant sans les redémontrer les justifications déjà posées précédemment.

4.1. Mise en place de l'environnement

Rappel du fonctionnement de Laravel. Laravel suit une architecture MVC classique, organisée autour d'un cycle de traitement de requête commun à toute application du framework. Une requête HTTP entrante est d'abord interceptée par le routeur, qui l'associe à une action définie dans un fichier de routes. Avant d'atteindre le contrôleur, la requête traverse une pile de middlewares : ce sont eux qui gèrent, entre autres, l'authentification — la session Laravel côté web, le middleware `CheckAccessToken` côté mobile (cf. 4.4). Le contrôleur reçoit ensuite la requête déjà authentifiée, orchestre l'accès aux modèles Eloquent (la couche métier, cf. 4.2) et produit une réponse — une vue Inertia côté web, un JSON encapsulé côté API (cf. 3.6).

L'autorisation proprement dite s'insère à un niveau distinct de ce cycle : ni dans le routage, ni dans le middleware, mais au sein même du contrôleur, via les Politiques Laravel — une classe dédiée par modèle, invoquée explicitement (`$this->authorize(...)`) pour déterminer si l'utilisateur authentifié a le droit d'effectuer l'action demandée sur la ressource ciblée. C'est ce mécanisme standard que le projet vient exploiter pour y brancher le calcul de permissions propre aux Training Resources (cf. 4.3).

Cette organisation logique se retrouve directement dans l'arborescence du projet, qui suit la structure standard d'une application Laravel sans réorganisation particulière. Le dossier `app/` concentre l'essentiel de la logique métier : les modèles Eloquent, les contrôleurs, les middlewares et les Politiques. Les routes sont déclarées dans `routes/`, séparées entre `web.php` et `api.php` comme évoqué plus haut. Le dossier `database/` porte les migrations et les seeders.

Côté front, Inertia fait cohabiter le frontend Vue dans la même base de code plutôt que dans un dépôt séparé : les composants vivent sous `resources/js`, organisés en pages (une par route Inertia) et composants réutilisables. La stack s'appuie sur Vue 3 et TypeScript, Vite comme bundler (via `laravel-vite-plugin`, qui l'intègre au cycle de build Laravel), Tailwind CSS 4 pour le style, et `shadcn-vue` comme socle de composants d'interface. Quelques bibliothèques spécialisées viennent compléter cette base pour des besoins ponctuels de l'interface — calendrier (FullCalendar), graphiques (ApexCharts) — sans remettre en cause cette organisation générale.

Outillage. Le projet s'appuie sur une configuration Laravel standard (MariaDB, sessions stockées en base) et sur Laravel 12 / PHP 8.2. L'authentification combine Laravel Sanctum avec le système de tokens custom du runner. L'intégration front repose sur `inertiajs/inertia-laravel`, complété par `inertiaui/modal` pour la gestion des modales côté Inertia et par `tightenco/ziggy`, qui expose les routes Laravel nommées côté JavaScript — évitant de dupliquer la définition des URLs entre le routeur PHP et le frontend Vue.

La documentation de l'API est générée automatiquement via `dedoc/scramble`, à partir des contrôleurs et des règles de validation existants plutôt que d'une spécification écrite séparément — un choix qui garantit que la documentation reste synchronisée avec le code réel plutôt que de dériver

au fil des évolutions (annexe 10.3). En développement, `laravel/pail` (suivi des logs en temps réel dans le terminal) et `barryvdh/laravel-debugbar` facilitent le débogage, et `laravel/pint` assure un style de code PHP homogène.

4.2. Implémentation du modèle métier

Migrations et structure des tables — hiérarchie des acteurs. Les tables `organizations`, `coaches` et `users` reprennent directement la hiérarchie décrite en 3.3. `coaches.organization_id` est une clé étrangère obligatoire en `CASCADE` : un coach ne peut exister sans organisation, et sa suppression entraîne celle de ses coachs. Le runner — modélisé par la table `users` — porte au contraire une double relation : `organization_id` obligatoire, mais `coach_id` nullable. C’est cette asymétrie qui matérialise en base la règle métier : la suppression d’un coach ne supprime jamais ses runners, elle les détache simplement en vue d’une réaffectation, alors que la suppression d’une organisation entraîne bien la suppression de tout ce qui lui appartient.

Le statut admin est porté indépendamment par `organizations.is_admin` et `coaches.is_admin`.

Propriété des Training Resources. `block_types`, `session_types`, `training_plans`, `training_session_templates` et `session_block_templates` partagent toutes la même paire de colonnes : `coach_id` et `organization_id`. Cette paire est la traduction directe en base du modèle de propriété à deux niveaux décrit en 3.4 — une ressource appartient soit à un coach précis, soit directement à son organisation. `session_block_templates` suit exactement le même schéma que `training_session_templates`.

Chacune de ces tables porte également une colonne `share`, qui ne sert aujourd’hui qu’à un seul cas d’usage : permettre à un coach admin de créer une ressource volontairement non visible, non copiable et non utilisable par les coachs classiques.

Composition d’un plan et polymorphisme. 3.3 pose que la structure interne d’un plan (séances, blocs) est la même, qu’il s’agisse d’un plan-modèle ou d’un plan assigné. `session_blocks` traduit cette exigence par une relation polymorphique — `sessionable_id / sessionable_type` — mais va plus loin que les deux cas envisagés en conception : en pratique, elle pointe vers trois types d’entités, `training_sessions` (séance d’un plan-modèle), `user_training_sessions` (séance assignée), et `training_session_templates` — une séance-type peut donc elle aussi porter directement sa propre composition de blocs, avec la même structure que les deux autres. La même table `session_blocks` sert ainsi indifféremment aux trois contextes sans duplication de structure.

L’imbrication des blocs (cf. 3.3, besoin des accélérations ponctuelles) est portée par un `parent_id` auto-référencé sur la même table.

Le contenu concret d’un bloc suit une séparation en trois temps, détaillée en 4.2 : `block_type_fields` définit, pour un `BlockType` donné, quels champs sont pertinents ; `block_fields` porte la valeur planifiée par le coach pour un bloc précis, avec son unité (`unit_id`) ; `block_results` porte enfin le résultat réellement exécuté par le runner (durée, distance, index, réussite), distinct de la valeur planifiée.

Synchronisation hors ligne. `session_blocks`, `block_fields` et `user_training_sessions` portent chacune un couple `deleted_at` (soft delete standard) / `is_restored` (booléen). C’est l’implémentation du mécanisme de restauration décrit en 3.7 : une suppression n’efface jamais la ligne, et `is_restored` garde une trace explicite qu’une entité supprimée a été réactivée plutôt que recrée — une nuance qu’un simple `deleted_at` remis à `NULL` ne suffirait pas à exprimer.

Authentification du runner. La table `tokens` porte les deux premiers des trois tokens décrits en 3.5 : un type `ACTIVATION` ou `ACCES` (enum), avec `expires_at`, `used_at` et `revoked_at` — ce dernier couple permet à la fois la rotation à usage unique du token d'accès (`used_at` renseigné dès sa présentation, cf. 4.4) et une révocation manuelle indépendante de l'expiration naturelle. Le troisième token, le token de session, ne vit pas dans cette table : il s'agit d'un `personal_access_tokens` Sanctum standard, généré à chaque échange à partir d'un token d'accès valide.

DIAGRAMME MCD/MLD — schéma relationnel complet des tables décrites ci-dessus (cf. annexe 10.2).

Modèles Eloquent et relations : rappel rapide du fonctionnement d'un modèle Eloquent.

Un modèle Eloquent représente une table (`protected $table`) et sert d'interface orientée objet vers ses lignes, sans que le développeur écrive de SQL pour les opérations courantes. Deux mécanismes protègent et interprètent les données à l'entrée comme à la sortie : `$fillable` liste les colonnes qu'on peut renseigner en masse (`Model::create([...])`, `->fill([...])`) — une protection contre l'assignation accidentelle de colonnes sensibles à partir d'une requête utilisateur — et `$casts` convertit automatiquement une colonne brute (un entier, une chaîne) vers un type PHP plus riche à la lecture, et inversement à l'écriture : `'share' => 'bool'` transforme le `tinyint(1)` de `training_plans.share` en véritable booléen PHP, `'training_type' => TrainingType::class` transforme la colonne `enum` en cas d'énumération PHP typée plutôt qu'en simple chaîne.

Les relations (`belongsTo`, `hasMany`, `hasOne`, `belongsToMany` ...) déclarent, sous forme de méthodes, comment une table se rattache à une autre — chacune correspondant à une clé étrangère ou une table pivot du schéma. Un cas plus spécifique au projet : les relations polymorphiques (`morphTo` / `morphMany`), utilisées quand une même table peut appartenir à plusieurs types de parents différents sans dupliquer sa structure.

Le polymorphisme de `SessionBlock`. `SessionBlock::sessionable()` illustre directement ce cas : un bloc appartient à un `sessionable_id` / `sessionable_type`, qui peut désigner indifféremment une `TrainingSession`, une `UserTrainingSession` ou une `TrainingSessionTemplate`. Plutôt que de laisser Eloquent stocker le nom de classe PHP complet dans `sessionable_type` — ce qui casserait au moindre renommage de classe — le projet déclare un `morphMap` explicite dans `AppServiceProvider::boot()`, qui associe chaque valeur de l'enum `SessionableType` à sa classe réelle. La colonne ne contient donc jamais que `training_session`, `user_training_session` ou `training_session_template`, découplant le nom stocké en base du nom de la classe PHP.

`TrainingSession::blocks()`, à l'inverse, est un `morphMany` filtré (`whereNull('parent_id')`) : il ne récupère que les blocs racines d'une séance, laissant les blocs enfants (imbrication, cf. 3.3/4.2) accessibles uniquement via `SessionBlock::children()`, une relation `hasMany` classique auto-référencée sur `parent_id`.

Relations traversantes : `hasOneThrough`. `TrainingSession::organization()` et `TrainingSession::coach()` utilisent `hasOneThrough`, une relation qui traverse une table intermédiaire sans clé étrangère directe : une séance n'a pas de `coach_id` / `organization_id` propre, seulement un `training_plan_id` — c'est donc `TrainingPlan` qui, lui, porte réellement `coach_id` / `organization_id`. `hasOneThrough` évite ainsi de charger le plan intermédiaire explicitement (`$session->trainingPlan->coach`) pour n'accéder qu'à un attribut final.

Cascades applicatives via les hooks du cycle de vie. Au-delà des cascades déclarées dans les migrations, certains modèles définissent des cascades applicatives dans leur méthode `booted()`, appelée automatiquement à chaque événement du cycle de vie du modèle (`saved`,

`deleting`, `forceDeleting`...). `SessionBlock::booted()` en est l'exemple le plus riche : à la suppression d'un bloc, il propage explicitement la suppression à ses enfants, ses champs planifiés, ses options et ses exercices — une cascade nécessaire dès lors que ces suppressions doivent rester des soft deletes cohérents avec la logique de synchronisation. Une seconde méthode, déclenchée sur `forceDeleting`, propage cette fois la suppression définitive. Seules les ressources d'un plan assigné (`UserTrainingSession` / `UserTrainingPlan`, synchronisées avec le mobile) ont besoin d'un soft delete pour rester cohérentes avec cette même logique — celles d'un plan-modèle (le “moule” côté coach, jamais transmis au mobile) n'ont aucune raison d'être conservées après suppression, et peuvent donc être supprimées directement. Code complet du modèle `SessionBlock` : cf. annexe 10.7 (classe `SessionBlock`).

Ce même mécanisme sert surtout à faire remonter un `touch()` le long de la chaîne des entités jusqu'à `user_training_plan` : `static::saved()` déclenche un `touch()` sur la `sessionable` du bloc, qui répercute lui-même sa propre mise à jour vers son plan assigné. Ce `updated_at` remonté est ce qui permet au mobile de savoir qu'une resynchronisation est nécessaire — l'application conserve un timestamp `last_synced` local, et compare simplement ce timestamp à celui du plan assigné dès qu'une connexion réseau redevient disponible, sans avoir à télécharger le plan pour le savoir. Le détail de ce mécanisme de synchronisation est développé en 4.4 (implémentation de l'API).

DIAGRAMME DE SÉQUENCE — propagation du `touch()` depuis `SessionBlock` jusqu'à `UserTrainingPlan` (cf. annexe 10.1).

Représentation d'un bloc : le pattern Entity-Attribute-Value. La composition d'un bloc suit un pattern EAV (Entity-Attribute-Value) : plutôt que de fixer, pour chaque type de bloc, un jeu de colonnes propre sur `session_blocks` (une colonne `distance`, une colonne `duration`, une colonne `intensity` ...), le schéma des attributs est lui-même une donnée. `block_field_types` définit le catalogue des attributs possibles (`DURATION`, `DISTANCE`, `REPETITION`, `INTENSITY`...), `block_type_fields` associe à un `BlockType` donné le sous-ensemble d'attributs qui le concerne, et `block_fields` porte enfin la valeur concrète (`data`) de chaque attribut pour une instance de bloc précise. Trois niveaux — catalogue, schéma, valeur — plutôt qu'une table unique et rigide.

Pourquoi ce choix plutôt qu'une alternative. Deux options plus simples ont été écartées :

- des colonnes fixes sur `session_blocks` auraient obligé à prévoir à l'avance l'union de tous les attributs possibles pour tous les types de blocs existants et à venir, avec une majorité de colonnes `NULL` pour chaque bloc (un bloc de récupération n'a pas de répétition, un bloc d'intervalle n'a pas les mêmes champs qu'un bloc de renforcement en `CONDITIONING`) ;
- une colonne JSON unique aurait évité cette dispersion, mais au prix de perdre la possibilité de contraindre un attribut par une clé étrangère (`block_fields.unit_id` référence `units`, garanti par une contrainte en base) et de devoir revalider manuellement la structure du JSON à chaque écriture, sans garantie native du SGBD.

L'EAV, ici, déplace la variabilité du schéma des colonnes vers les lignes : ajouter un nouveau type d'attribut ne nécessite ni migration ni redéploiement, seulement une ligne supplémentaire dans `block_field_types` — un avantage direct compte tenu du rythme d'évolution du catalogue de blocs observé dans l'historique des migrations (ajouts successifs comme `training_type` ou le système d'unités).

Contreparties assumées. Ce choix a un coût :

- **Requêtes multiples plutôt qu'une lecture directe.** Afficher un bloc avec tous ses champs nécessite de charger beaucoup de relations/jointures en cascade (visible dans

`SessionBlock::scopeWithRelations()`). Un `SELECT * FROM session_blocks` ne suffit jamais à lui seul, contrairement à un schéma à colonnes fixes.

- **Validation déportée à l'application.** Rien en base n'empêche qu'un `block_field` référence un `block_type_field_id` qui n'appartient pas au `BlockType` réel du bloc — c'est au code applicatif de garantir cette cohérence, la contrainte n'étant pas structurellement exprimable par une clé étrangère simple.
- **Typage de la donnée appauvri.** `block_fields.data` est un entier générique (`int(11)`) quel que soit l'attribut réel — une distance, une durée, une répétition partagent la même colonne, la sémantique du type dépendant entièrement du `block_field_type` associé plutôt que d'un typage SQL natif par colonne.

Migrations des tables `block_field_types` / `block_type_fields` / `block_fields` : cf. annexe 10.8 (migrations EAV).

4.3. Implémentation des Training Resources

Les interfaces en code. La classe abstraite `TrainingResource` implémente simultanément six interfaces — `Editable`, `Ownable`, `Deletable`, `Copiable`, `Useable`, `Viewable` — traduction directe en PHP du découpage par comportement décrit en 3.4. Chaque interface capture une question isolée (peut-on éditer ? copier ? utiliser ?), et c'est leur combinaison sur une même classe qui donne à `TrainingResource` sa surface complète.

L'accès au propriétaire, lui, repose sur trois traits combinables, `HasCoach`, `HasOrganization`, `HasUser`. Chacun expose une paire cohérente : une relation Eloquent (`coach()`, `organization()`, `user()`) et un accesseur d'identifiant qui tolère deux représentations différentes de la même donnée. `HasCoach::coachId()` en est l'exemple le plus parlant : il retourne d'abord l'attribut brut `coach_id` s'il est chargé, et ne déclenche la relation (`$this->coach?->id`) que si l'attribut n'est pas directement présent sur le modèle — un besoin concret, puisque `TrainingSession` n'a justement pas de `coach_id` propre et ne peut retrouver son coach qu'en traversant `trainingPlan` via `hasOneThrough`. Le même principe s'applique à `HasOrganization::organizationId()`. Ce découplage entre “comment la donnée est stockée” et “comment on y accède” est ce qui permet à `basePermissionsEdit` / `basePermissionRead`, écrites une seule fois dans la classe de base, de fonctionner indifféremment sur une ressource à propriété directe (`TrainingPlan`) ou indirecte (`TrainingSession`).

`UserTrainingResource` étend `TrainingResource` et ajoute `HasUser` à ce même principe, pour les ressources assignées (cf. 3.4) — une quatrième relation de propriété (`user()`), traitée exactement comme les deux précédentes. Code complet de `TrainingResource`, `UserTrainingResource` et des traits `HasCoach` / `HasOrganization` / `HasUser` : cf. annexe 10.9 (classe `TrainingResource`).

DIAGRAMME DE CLASSES — hiérarchie `TrainingResource` / `UserTrainingResource`, ses interfaces (`Editable`, `Ownable`, `Deletable`, `Copiable`, `Useable`, `Viewable`) et les traits `HasCoach` / `HasOrganization` / `HasUser` (cf. annexe 10.1).

Calcul des permissions. `TrainingResource::permissions()` centralise en un seul point ce que 3.4 décrivait comme “la même méthode calcule le même ensemble de droits pour n'importe quelle ressource” : elle appelle `canEdit`, `canDelete`, `canCopy`, `canUse`, `canView` sur l'instance courante et assemble le résultat dans un tableau indexé par l'enum `ResourcePermissions` — c'est ce tableau, sérialisé, que consomme directement le frontend pour afficher ou masquer les actions disponibles (cf. 4.5). Un détail à noter : `COPY` et `USE` sont conditionnées en plus par `$user instanceof Coach`

— une organisation, en tant qu’actrice, ne copie ni n’utilise directement une ressource, seul un coach le fait.

`isAdminResource()` mérite une remarque : elle compare `organization.name` à `config('app.admin_organization_name')` plutôt que de lire un champ `is_admin` déjà chargé sur l’organisation propriétaire — une comparaison par nom recalculée à chaque appel, plutôt qu’une lecture directe du booléen déjà porté par `Organization`. Les deux reviennent au même résultat puisque l’organisation admin est unique, mais c’est une redondance à signaler plutôt qu’à statuer soi-même.

`UserTrainingResource::basePermissions()` remplace entièrement cette logique par une seule règle — `$user->isCoachOf($this->user)` — pour une ressource assignée, la propriété au sens coach/organisation ne s’applique plus, seule compte la relation d’encadrement entre le coach consultant et le runner concerné. Une organisation, elle, ne peut jamais rien sur une ressource assignée (`return false` immédiat) — l’assignation est une relation exclusivement coach-runner.

Les Politiques Laravel. Le projet type son autorisation sur un contrat `Identifiable` plutôt que sur les classes Eloquent concrètes de Laravel — `getOrganizationId()`, `isAdmin()`, `role()`, `getId()`, `getName()` — implémenté à la fois par `Coach` et `Organization`. C’est ce contrat qui permet aux deux Politiques (`TrainingResourcesPolicy`, `UserTrainingResourcePolicy`) de rester génériques, sans distinguer si l’acteur authentifié est un coach ou une organisation : chaque méthode de Policy (`view`, `update`, `delete`, `copy`, `use`) ne fait que déléguer à la méthode `can*`() correspondante déjà définie sur le modèle — la Policy ne réimplémente aucune règle, elle sert uniquement de point d’entrée standard Laravel vers une logique déjà écrite une seule fois.

Ce report de logique a une conséquence directe sur `Controller::authorize()`, qui **surcharge** la méthode standard fournie par le trait `AuthorizesRequests` : plutôt que de s’appuyer sur `Auth::user()` (le guard Laravel classique, pensé pour un unique modèle `User` authentifié), elle invoque `authorizeForUser($this->currentUser(), ...)` — `currentUser()` retournant un `Identifiable`, résolu par un service d’authentification dédié (`WithAuthService`) capable de distinguer une session coach/organisation d’un token runner (cf. 3.5). C’est ce point précis qui permet aux mêmes Politiques de fonctionner indifféremment que l’acteur authentifié soit un coach, une organisation, ou — via `UserTrainingResourcePolicy` — un coach agissant sur une ressource d’un runner.

Code complet de `TrainingResourcesPolicy` et `UserTrainingResourcePolicy` : cf. annexe 10.10 (Politiques des Training Resources).

DIAGRAMME DE SÉQUENCE — trajet d’une requête contrôleur autorisée : `Controller::authorize()` → `authorizeForUser()` → `TrainingResourcesPolicy` → `TrainingResource::can*()` (cf. annexe 10.1).

4.4. Implémentation de l’API

Routing et séparation web/mobile. `routes/api.php` distingue nettement deux niveaux d’accès, cohérents avec 3.5 : un premier groupe de routes reste public — `/responses`, `/questions` (le questionnaire pour récupérer un plan sans coach) et `/users/activate` (l’étape d’activation elle-même, qui ne peut évidemment pas exiger le token qu’elle a justement pour rôle de délivrer) — tandis que le reste est rassemblé sous un préfixe `user/` et protégé par `CheckAccessToken::class`. Toute la déclaration est en outre enveloppée dans un middleware `Cors`, nécessaire puisque l’application mobile consomme cette API depuis un contexte différent du navigateur web.

Le middleware `CheckAccessToken`. Ce middleware implémente exactement le mécanisme des

tokens décrit en 3.5, en gérant les deux cas possibles selon ce que le client présente en en-tête `Authorization` (code complet : cf. annexe 10.11, middleware `CheckAccessToken`) :

Cas 1 — un token de session encore valide. Le middleware tente d'abord de résoudre le bearer token comme un `PersonalAccessToken Sanctum` (`PersonalAccessToken::findToken($token)`). S'il existe et que `expires_at` n'est pas dépassé, la requête continue immédiatement : `last_used_at` est rafraîchi, et le résolveur d'utilisateur de la requête (`$request->setUserResolver`) est fixé sur le runner propriétaire du token — c'est ce qui alimente ensuite `$request->user()` dans les contrôleurs (cf. `TrainingPlanAPIController::index`). Aucune rotation n'a lieu dans ce cas : le token de session, valide 15 minutes (cf. 3.5), sert tel quel jusqu'à son expiration.

Cas 2 — pas de session valide, bascule sur le token d'accès. Si le token de session est absent ou expiré, le middleware réinterprète le même bearer token comme un token d'accès custom, recherché en base par son hash SHA-256 (`Token::where('token', hash('sha256', $token))->where('type', TokenType::CUSTOM)`) — la table `tokens` ne stocke donc jamais le token en clair, seulement son empreinte. `TokenStatus::fromToken()` centralise la validation (existence, expiration, révocation, déjà utilisé) et renvoie un statut typé, permettant à `tokenError()` de répondre une erreur adaptée sans que le middleware n'ait à enchaîner une série de `if` redondants.

Si le token d'accès est valide, trois choses se produisent dans le même passage : un nouveau token de session Sanctum est généré (`$user->createToken(...)`), durée pilotée par `config('token.session.ttl')`, l'ancien token d'accès est marqué utilisé (`markUsed()`) plutôt que supprimé, et un nouveau token d'accès est généré immédiatement (`Token::generateAccessToken($user)`) — c'est cette dernière étape qui matérialise la rotation glissante décrite en 3.5 : chaque présentation du token d'accès en produit un nouveau, prolongeant la fenêtre de 14 jours à chaque usage réel plutôt que de la laisser courir depuis l'activation initiale. Les deux nouveaux tokens (session et accès) ne sont pas renvoyés dans le corps de la réponse, mais en en-têtes dédiés — `X-Access-Token` et `X-Session-Token` — après exécution de la requête elle-même (`$next($request)`), pour que le client mobile les récupère et les stocke sans polluer le payload métier de la réponse.

DIAGRAMME DE SÉQUENCE — les deux cas de `CheckAccessToken` (session valide / rotation via token d'accès), à mettre en regard du diagramme d'authentification déjà posé en 3.5 (cf. annexe 10.1).

Contrôleurs de l'API mobile — synchronisation incrémentale. `php TrainingPlanAPIController::index()` ne renvoie pas systématiquement tous les plans assignés au runner (cf. 3.6), mais uniquement ceux modifiés depuis un paramètre `since` (`->where('updated_at', '>', $since)`), avec la date de création du compte comme valeur par défaut si le mobile n'en fournit pas. C'est ici que se referme la boucle amorcée en 4.2 : le `touch()` qui remonte depuis un `SessionBlock` jusqu'à `UserTrainingPlan` met à jour ce même `updated_at` que ce endpoint compare — le mobile envoie son `last_synced` local en `since`, et ne reçoit que le ou les plans qui ont réellement changé depuis, sans avoir à retélécharger l'intégralité de ses plans à chaque appel. La réponse porte elle-même un `timestamp`, produit par le serveur (`now()->toDateTimeLocalString()`) plutôt que calculé côté client : c'est cette valeur que le mobile enregistre comme nouveau `last_synced`, et non l'heure de son propre horloge au moment de l'appel — une précaution qui évite tout décalage entre les deux appareils, qu'il vienne d'une horloge mal synchronisée ou du temps réseau écoulé pendant la requête.

Les plans renvoyés et actuellement en cours ou à venir (`TrainingPlanStatus::ONGOING / UPCOMING`) sont ensuite marqués `pulled()` — l'écriture du `pulled_at` déjà mentionné en 3.3, qui trace côté serveur le moment où le mobile a effectivement récupéré un plan.

Résultats et restauration en cascade. `TrainingSessionApiController::storeResults()` illustre concrètement le mécanisme de synchronisation hors ligne détaillé en 3.7. Le contrôleur recharge explicitement la séance `withTrashed()` : si elle avait été supprimée logiquement (par exemple un runner ayant supprimé puis rétabli une séance côté mobile pendant une période hors ligne), elle est restaurée et marquée `is_restored`. La même logique de restauration en cascade se répète au niveau du bloc concerné par chaque résultat (`$b->trashed() → restore()` sur le bloc, ses champs, ses options, ses exercices) — la restauration en cascade mentionnée dans le contexte du projet, ici rendue explicite plutôt que centralisée (le risque de capture dispersée déjà identifié en 3.7). Le garde-fou anti-doublon apparaît sous la forme `$b->results->isEmpty()` : un résultat n’est créé que si le bloc n’en possède pas déjà un, empêchant qu’un même résultat soit enregistré deux fois si le mobile rejoue un envoi après une coupure réseau.

Un point métier s’ajoute à cette mécanique : si la séance est la séance-type protégée du test de VMA (`$session->sessionType->isVmaSession()`, cf. 4.2), les résultats fraîchement enregistrés déclenchent un recalcul automatique de la VMA (`VmaService::vmaAlgoJimmy()`), aussitôt répercuté sur `UserTrainingPlan` — la traduction concrète du calcul algorithmique évoqué en 3.4, qui justifiait la non-modifiabilité de cette séance-type. Code complet de `TrainingPlanApiController` et `TrainingSessionApiController` : cf. annexe 10.12 (contrôleurs API mobile).

DIAGRAMME DE SÉQUENCE — `storeResults()` : restauration en cascade (séance → bloc → champs/options/exercices), garde-fou anti-doublon, puis recalcul VMA conditionnel (cf. annexe 10.1). Exemple de payload envoyé par le mobile et de réponse retournée : cf. annexe 10.6.

Deux usages différents du signal de synchronisation. `storeResults()` termine par un appel explicite à `$plan->pushed()`, encapsulé dans `UserTrainingPlan::withoutTimestamps(...)` — cette dernière précaution évite qu’écrire `pushed_at` ne redéclenche, via le cycle de vie normal du modèle, une mise à jour de `updated_at` sur le plan lui-même, ce qui fausserait justement le signal de synchronisation décrit plus haut. À l’inverse, `saveReview()` encapsule sa propre écriture dans `UserTrainingSession::withoutTouchingOn([UserTrainingPlan::class], ...)` : un feedback qualitatif du runner n’a pas besoin de faire remonter un signal “resynchronise-toi” jusqu’au plan, puisque c’est le runner lui-même qui vient d’écrire cette donnée — contrairement à un résultat de séance, qui doit au contraire remonter jusqu’au coach côté web.

Documentation générée. Cf. 4.1 : la documentation de l’API mobile est générée automatiquement par Scramble à partir de ces mêmes contrôleurs et `FormRequests`, sans spécification écrite séparément — les annotations `@tags` visibles dans les deux contrôleurs (`Training Sessions`, `Training Plans`) organisent directement le regroupement des endpoints dans la documentation produite. Documentation complète générée : cf. annexe 10.3. Capture d’écran de l’interface Scramble : cf. annexe 10.5.

4.5. Interface utilisateur

Inertia/Vue en pratique. Le flux suit exactement le principe posé en 3.2 : le contrôleur reste seul maître de l’état, sans exposer d’API JSON publique dédiée au web. Entre le modèle Eloquent et la page Vue s’intercale une Laravel Resource (`JsonResource`) — une classe dont l’unique rôle est de transformer un modèle en tableau prêt à sérialiser, indépendamment de sa structure de colonnes réelle en base : elle choisit quels champs exposer, sous quel nom, avec quelles relations imbriquées, sans jamais renvoyer le modèle Eloquent brut. `UserTrainingSessionResource::toArray()` en est un exemple direct : elle transforme une `UserTrainingSession` en un tableau `{id, type, permissions, sessionable, sessionBlocks, date, comment, fee`

calculant au passage certains champs à la volée — `'permissions' => $this->permissions($coach)` appelle directement la méthode du modèle vue en 4.3, et `'date'` résout la date absolue de la séance à partir de la date de début du plan (cf. 3.6). `TrainingSessionController::show()` charge les relations nécessaires, passe le modèle à travers cette couche de sérialisation, puis transmet le résultat au frontend via Inertia.

Le composant Vue de la page (`Show.vue`) déclare ces mêmes données en `defineProps<{...}>()`, typées côté frontend par les interfaces `TrainingPlan`, `TrainingSession`, etc. (fichier `types.ts`) — un typage qui reflète la structure produite par la Resource backend.

Capture d'écran de la page `coach/trainingSessions/Show` : cf. annexe 10.5.

Exploitation des permissions côté front. Le tableau `permissions` calculé par `TrainingResource::permissions` (cf. 4.3) et sérialisé par chaque Resource (`UserTrainingSessionResource` pour une séance, un mécanisme identique pour un plan ou un bloc) devient, côté Vue, un objet typé `Permissions { edit, use, copy, delete` porté par chaque ressource reçue en props. C'est ce même objet, jamais recalculé côté client, qui pilote entièrement l'affichage des actions disponibles.

`ActionMenu.vue` illustre bien la conséquence de ce choix : le composant lui-même ignore complètement ce qu'est une Training Resource — il ne reçoit qu'un tableau `actions: MenuAction[]`, chaque action portant déjà son propre attribut `disabled: boolean`, et se contente de rendre un menu accessible sans jamais consulter les permissions lui-même. C'est le composant appelant qui construit ce tableau en lisant les permissions de la ressource concernée :

- au niveau d'un plan (`Show.vue`), `actions` teste `props.trainingPlan.permissions.edit / .copy / .delete` pour désactiver “Edit infos”, “Copy plan” ou “Delete plan” ;
- au niveau d'une séance ou d'un bloc sélectionné (`SectionSelectedSession.vue`), le même composant `ActionMenu` reçoit un tableau différent — `actionsMore.concat(actions)` — où `actions` est recalculé localement à partir de `session.permissions`, avec une nuance supplémentaire : la disponibilité de “Delete block” dépend en plus de `sessionHasResults(session)`, une règle qui ne relève pas de la propriété ou du partage (cf. 3.4) mais d'une contrainte fonctionnelle propre à l'interface (on ne supprime pas un bloc dont un résultat a déjà été enregistré).

Ce découplage — un composant de menu générique, une logique de permission entièrement déportée vers l'appelant — permet de réutiliser `ActionMenu` à n'importe quelle granularité (organisation, plan, séance, bloc) sans jamais dupliquer sa mécanique d'affichage, tout en gardant la source de vérité des droits côté serveur. Code complet d' `ActionMenu.vue` : cf. annexe 10.13 (composant `ActionMenu`).

Capture d'écran d' `ActionMenu` avec une action désactivée (coach non propriétaire) : cf. annexe 10.5.

Captures d'écran du panneau latéral en mode séance et en mode bloc sélectionné : cf. annexe 10.5.

5. Sécurité et protection des données

Cette partie rassemble et synthétise les mécanismes de sécurité et de confidentialité déjà présentés en détail dans les parties précédentes, plutôt que de les redémontrer. Certains points, en revanche, n'ont encore été traités nulle part ailleurs — ils sont développés ici, y compris lorsqu'il s'agit d'assumer une limite actuelle du produit plutôt que de la maquiller.

5.1. Principes de confidentialité et minimisation des données

Le principe de minimisation qui structure ce travail trouve son origine dans le concept fondateur du produit (cf. 1.1) et se retrouve formalisé comme besoin non fonctionnel explicite (cf. 2.4). Sa traduction concrète — pseudonyme du runner, absence de transmission GPS, payload d'écriture volontairement minimal côté API — est déjà entièrement démontrée en 3.3 et 3.6 ; ce principe ne fait donc l'objet d'aucun développement technique supplémentaire ici.

5.2. Sécurité de l'authentification

La sécurité du système à trois tokens du runner (empreinte SHA-256 en base, jamais de token en clair, rotation glissante) est déjà entièrement couverte en 4.4 et n'est pas redéveloppée. Un point n'a en revanche pas encore été montré : la politique de mot de passe appliquée aux comptes coach et organisation, définie globalement dans `AppServiceProvider::boot()` :

```
Password::defaults(function () {  
    return Password::min(12)  
        ->letters()  
        ->mixedCase()  
        ->numbers()  
        ->symbols()  
        ->uncompromised();  
});
```

`Password::defaults()` s'applique par défaut à toute règle de validation `password` du projet, sans que chaque `FormRequest` n'ait à la redéfinir. Au-delà des exigences classiques (longueur minimale, casse mixte, chiffres, symboles), `->uncompromised()` mérite une remarque : Laravel interroge l'API *Have I Been Pwned* pour vérifier que le mot de passe choisi ne figure pas parmi les mots de passe déjà exposés lors de fuites de données connues — une protection contre la réutilisation de mots de passe compromis, plutôt qu'une simple contrainte de format. C'est aujourd'hui la seule politique de mot de passe en place ; aucune authentification à deux facteurs ni expiration périodique de mot de passe ne sont mises en œuvre.

5.3. Isolation des données

L'isolation entre organisations découle structurellement du modèle de propriété décrit en 3.4 et implémenté en 4.3 : `basePermissionsEdit()` / `basePermissionRead()` comparent systématiquement l'organisation de l'utilisateur à celle de la ressource, rendant impossible l'accès d'un coach aux données d'une organisation à laquelle il n'appartient pas. Ce mécanisme n'est pas une brique de sécurité séparée, mais une conséquence directe du calcul de permissions déjà démontré ; il n'est donc pas redéveloppé ici.

5.4. Sécurisation de l'API

La validation des données entrantes via les `FormRequest` et l'usage de `$fillable` pour se prémunir de l'assignation de masse sont déjà présentés en 4.2 et 4.4. Le CORS repose sur une configuration standard, sans règle spécifique au projet méritant d'être détaillée.

Limite actuelle. Aucun rate limiting n'est en place à ce stade sur les routes sensibles de l'API (activation de token, tentative de connexion) : rien n'empêche aujourd'hui un enchaînement rapide de requêtes sur ces endpoints. C'est une limite assumée du produit en l'état actuel plutôt qu'un choix de conception, reprise en perspective d'évolution (cf. 8.3).

5.5. RGPD et Privacy by Design

Le seul principe RGPD effectivement mis en œuvre à ce jour est la minimisation des données, déjà démontrée à plusieurs reprises (cf. 3.3, 5.1). Aucun mécanisme de droit à l'effacement ni d'export des données personnelles n'est aujourd'hui implémenté, et aucune politique de durée de conservation n'est formellement définie. Plutôt que de présenter une conformité RGPD que le produit n'atteint pas encore, ces éléments sont assumés comme limites actuelles de la solution, à traiter en priorité dans les perspectives d'évolution (cf. 8.3).

6. Tests, validation et mise en production

6.1. Stratégie de test. Aucun test automatisé n'existe à ce stade du projet. L'outillage est en place — pestphp/pest et pestphp/pest-plugin-laravel figurent déjà parmi les dépendances du projet — mais la suite de tests elle-même reste à écrire. Ce choix n'est pas un oubli mais une priorisation assumée : le produit étant encore en évolution rapide au contact de ses premiers utilisateurs réels (cf. 1.4), écrire une suite de tests avant que le comportement attendu ne se stabilise aurait représenté un investissement risquant d'être en grande partie à refaire à chaque ajustement issu des retours bêta. La validation repose aujourd'hui sur deux piliers non automatisés : la validation métier par le coach expert pour la structure des entraînements (cf. 1.4), et l'usage réel du produit par des clients en bêta comme test de terrain. L'écriture d'une suite de tests automatisés, une fois cette phase de stabilisation passée, est reprise comme perspective à court terme (cf. 8.3).

6.2. Scénarios de test représentatifs. En l'absence de suite automatisée (cf. 6.1), chaque déploiement est précédé d'une vérification manuelle qui rejoue le parcours complet du produit plutôt que des cas isolés — une manière de couvrir empiriquement l'essentiel de la chaîne fonctionnelle malgré l'absence de tests écrits :

Chaîne de création des acteurs. Création d'une organisation, puis d'un coach rattaché à celle-ci, puis d'un runner rattaché à ce coach — le parcours suit délibérément la hiérarchie organisation → coach → runner (cf. 3.3), puisqu'une régression à un niveau se répercuterait mécaniquement sur les suivants. Construction et utilisation des Training Resources. Création de ressources (types de blocs, types de séances...), puis leur réutilisation effective pour composer une séance et un plan — vérifie que le cycle création → composition fonctionne de bout en bout, pas seulement la création isolée d'une ressource. Assignment d'un plan. Attribution du plan construit à un runner, avec sa VMA — le point de jonction entre les Training Resources réutilisables et les entités assignées (cf. 3.3/3.4). Permissions. Vérification que les règles d'accès restent cohérentes tout au long de ce parcours (un coach ne peut agir que sur ce qui relève de son périmètre, cf. 3.4/4.3) — un point de vigilance particulier puisque les permissions sont calculées à la volée plutôt que stockées, et donc plus sensibles à une régression silencieuse qu'un champ en base resté correct par simple absence de modification.

Cette checklist reste manuelle et informelle — elle ne remplace pas une suite de tests automatisée, mais elle couvre le chemin critique du produit avant chaque mise en production, en cohérence avec la priorisation assumée en 6.1.

6.3. Mise en production. Deux environnements distincts coexistent, avec des objectifs différents plutôt qu'une simple gradation dev/prod classique. Un environnement de staging (dev.notrackrun.eu) tourne sur Laravel Octane avec FrankenPHP, dans le but précis de tester en conditions réelles la communication entre l'application mobile et la plateforme — un environnement d'intégration continue informel plutôt qu'un simple miroir de la prod. La production, elle, repose sur Docker Compose avec PHP-FPM classique, sans Octane : ce choix est délibéré et documenté — Octane, avec son modèle d'application persistante entre les requêtes (le serveur applicatif reste en mémoire plutôt que de redémarrer à chaque requête comme PHP-FPM), introduit des risques spécifiques autour de l'état partagé entre requêtes (singletons, état statique qui persisterait involontairement d'une requête à l'autre) qui ne sont pas encore jugés suffisamment maîtrisés pour être exposés aux clients réels. Octane reste donc cantonné à l'environnement de staging le temps de cette validation, la prod privilégiant la robustesse éprouvée de PHP-FPM.

7.1. Liens avec les cours du bachelier

Ce projet a été l’occasion de mobiliser de manière concrète les compétences acquises au fil du bachelier en Informatique de gestion.

Les cours de conception logicielle ont été les plus directement mobilisés. La construction des diagrammes UML — modèle conceptuel et logique de données, diagrammes de classes, diagrammes de séquence — ainsi que la conception de l’abstraction des Training Resources (interfaces comportementales, séparation entre modèle, Policy et logique de permission, cf. 3.4/4.3) reflètent directement les principes enseignés en matière de qualité logicielle et de maintenabilité du code. La décision de documenter explicitement une tentative de conception abandonnée — l’évent sourcing hybride (cf. 3.7) — plutôt que de ne présenter que la solution finale retenue s’inscrit dans la même logique : justifier un choix de conception suppose de pouvoir aussi expliquer pourquoi une autre option a été écartée.

Les cours de gestion de projets ont quant à eux servi de cadre pour la maîtrise de Git et du versioning, dans un contexte rendu plus exigeant par le travail à deux sur un même produit (cf. 1.3) : la coordination avec Alexandre Henin sur le contrat d’API entre plateforme et application mobile a nécessité une discipline de communication technique — documenter les évolutions d’un endpoint avant qu’elles n’impactent le développement mobile en parallèle — que la gestion d’un dépôt individuel n’impose pas de la même façon.

Les cours de bases de données ont guidé la conception du schéma de données, notamment la définition des relations entre entités, les règles de cascade (`ON DELETE CASCADE` / `SET NULL` selon la nature de la relation, cf. 4.2) et les choix de modélisation plus spécifiques au projet, comme le pattern EAV retenu pour la composition d’un bloc (cf. 4.2) — une solution qui dépasse le cadre du modèle relationnel classique vu en cours, mais qui n’aurait pas pu être évaluée à sa juste valeur sans cette base.

7.2. Difficultés rencontrées

La principale difficulté a été de faire évoluer un produit dont le besoin lui-même change en cours de route, plutôt que de développer face à un cahier des charges figé. Le second type d’entraînement orienté conditionnement (cf. 3.3) en est l’exemple le plus direct : une fonctionnalité demandée en cours de bêta, à intégrer dans un modèle déjà en production plutôt que pensée dès la conception initiale. Cette expérience a mis en évidence un piège concret : sur-abstraire trop tôt dans le développement se révèle souvent aussi coûteux que ne pas abstraire du tout. Une abstraction posée avant que le besoin ne soit suffisamment stabilisé risque de contraindre les évolutions futures dans une direction qui ne correspond plus à la réalité du produit, obligeant à la défaire ou à la contourner plutôt qu’à l’étendre naturellement — un équilibre qui s’apprend surtout par l’expérience, difficile à anticiper en théorie.

Une autre difficulté a tenu à la nécessité de concilier deux environnements aux contraintes très différentes — la plateforme web et l’application mobile — développés par deux personnes en parallèle (cf. 1.3). Le mobile n’est pas simplement un second client de la même API : il est soumis à des contraintes que le web ne connaît pas (connectivité intermittente, cf. 3.7 ; persistance locale des données ; cycle de vie de l’application différent d’une page web), ce qui a nécessité de penser certaines décisions de conception — comme le payload consolidé plutôt que fragmenté (cf. 3.6) — en fonction de contraintes qu’un développement purement web n’aurait jamais fait apparaître.

Faire évoluer la structure des modèles de données sans perdre les données déjà existantes en production a également représenté une difficulté récurrente. Une migration qui fonctionne bien sur une base vide se comporte différemment face à des données réelles déjà accumulées — il a fallu

apprendre à distinguer une migration de structure simple d'une migration nécessitant une étape de transformation des données existantes, avec le risque associé de perte ou d'incohérence si cette distinction est mal évaluée.

Enfin, une difficulté plus technique a émergé autour des performances : la facilité avec laquelle Eloquent peut générer des requêtes N+1 sans que cela soit visible au premier abord. Barryvdh/laravel-debugbar (cf. 4.1) a été l'outil déterminant pour identifier ces requêtes dupliquées, invisibles à la simple lecture du code mais immédiatement flagrantes une fois le nombre de requêtes réellement exécutées affiché — un rappel concret que l'eager loading (with(...)) n'est pas une optimisation facultative une fois la structure de données suffisamment imbriquée

7.3. Compétences acquises et apport professionnel

Ce projet a été l'occasion de dépasser un usage superficiel de Laravel pour aller vers une maîtrise plus fine du framework — le comportement d'Eloquent au-delà du CRUD basique (relations polymorphiques, hooks de cycle de vie, morphMap), l'articulation entre Politiques et logique métier plutôt que leur simple juxtaposition. Au-delà du framework, c'est la capacité à concevoir une abstraction réutilisable à partir d'un besoin métier réel, plutôt qu'à partir d'un exemple de cours, qui constitue l'apport le plus transférable professionnellement — avec la leçon associée qu'une bonne abstraction se construit progressivement, au contact du besoin réel, plutôt que d'être figée trop tôt (cf. 7.2).

La confrontation à des contraintes de production réelles — minimisation des données, tolérance au hors-ligne, coexistence de deux environnements de déploiement aux objectifs distincts (cf. 6.3) — a développé une sensibilité aux compromis d'ingénierie qu'un projet académique isolé développe rarement : chaque décision de conception documentée dans ce travail porte la trace d'un choix pesé entre plusieurs options, plutôt que d'une solution unique évidente.

Enfin, la coordination avec un développeur mobile sur un contrat d'API partagé (cf. 1.3, 3.6), la capacité à faire évoluer un modèle de données en production sans perdre l'existant, et le réflexe de vérifier les performances réelles plutôt que supposées (cf. 7.2) sont des compétences directement mobilisables en entreprise, indépendamment du framework ou du langage utilisé.